

Análise de Complexidade das Classes da USEI09

Classe 1: `ProximitySearchController`

Linha de Código	Complexidade	Explicação
<code>public ProximitySearchController() {</code>	$O(1)$	Construtor com atribuições constantes
<code> this.searcher = new NearestNeighborSearcher();</code>	$O(1)$	Criação de objeto
<code> this.stationRepository = StationRepository.getInstance();</code>	$O(1)$	Acesso ao singleton
<code>public List<StationDistance> findNearestStations(...)</code>	$O(\log k + n)$ médio $/ O(k)$ pior	Complexity do KD-tree
<code> KDtree kdtree = stationRepository.getKdtree();</code>	$O(1)$	Acesso a referência
<code> ` if (kdtree == null</code>		<code>kdtree.getRoot() == null)`</code>
<code> throw new IllegalStateException(...)</code>	$O(1)$	Criação de exceção
<code> return searcher.findNearestN(...)</code>	$O(\log k + n)$ médio $/ O(k)$ pior	Chamada ao algoritmo KD-tree
<code>public SearchResult findNearestStationsWithMetrics(...)</code>	$O(\log k + n) / O(k)$ pior	Idem com métricas
<code> createTimeZoneFilter(...)</code>	$O(1)$	Criação de filtro
<code> createTypeFilter(...)</code>	$O(1)$	Atribuições constantes
<code> getLastSearchNodesVisited()</code>	$O(1)$	Retorno simples

Classe 2: `StationFilter`

Linha de Código	Complexidade	Explicação
<code>public StationFilter()</code>	$O(1)$	Construtor simples
<code>withTimeZoneRange(...)</code>	$O(1)$	Atribuições
<code>matches(RailwayStation station)</code>	$O(1)$	Verificações booleanas
<code>if (station == null)</code>	$O(1)$	Check nulo
<code>TimeZoneGroup stationTimeZone = station.getTimeZoneGroup()</code>	$O(1)$	Getter
<code>isBetween(...)</code>	$O(1)$	Comparação
<code>isCity != null && station.isCity() != isCity</code>	$O(1)$	Booleano
<code>toString()</code>	$O(1)$	String limitada
<code>StringBuilder sb = ...</code>	$O(1)$	Construção constante

Classe 3: `StationDistance`

Linha de Código	Complexidade	Explicação
Construtor	$O(1)$	Inicialização
<code>if (station == null)</code>	$O(1)$	Check nulo
<code>if (distance < 0)</code>	$O(1)$	Check numérico
<code>compareTo</code>	$O(1)$ para double, $O(m)$ para strings	m = comprimento do nome
<code>equals</code>	$O(1) + O(m)$	Comparações simples e de strings
<code>hashCode</code>	$O(1) + O(m)$	Hash de string

Linha de Código	Complexidade	Explicação
toString()	O(1)	Formatação

Classe 4: `NearestNeighborSearcher`

Linha	Complexidade	Explicação
findNearestN(...)	O(log k + n) médio / O(k) pior	Algoritmo KD-tree
Checks iniciais	O(1)	Verificações simples
PriorityQueue criação	O(1)	Heap vazio
searchWithPruning(...)	O(log k + n) médio / O(k) pior	Busca recursiva
Criação de lista final	O(n)	Cópia de heap
result.sort(...)	O(n log n)	Ordenação
Logs / mensagens	O(1)	Constante
for station in node.getStations()	O(s)	s = estações no nó
haversineDistance	O(1)	Operações matemáticas
Heap offer/poll	O(log n)	Operações do heap
Escolha de subárvore	O(1)	Comparações
shouldExploreFarSubtree	O(1)	Distância ao plano
countMatchingStations(...)	O(k)	Percurso completo
findNearestNWithMetrics	O(log k + n) / O(k)	Busca + tempo

Classe 5: `DistanceUtils`

Linha	Complexidade	Explicação
<code>haversineDistance(...)</code>	$O(1)$	Cálculos matemáticos fixos
Converte graus $\rightarrow$ radianos	$O(1)$	Simples
Seno, cosseno, <code>atan2</code>	$O(1)$	Custo constante
<code>return EARTH_RADIUS_KM * c</code>	$O(1)$	Multiplicação
<code>distanceSquared(...)</code>	$O(1)$	Operações aritméticas simples

Análise de Complexidade Geral da USEI09

A implementação da USEI09 apresenta uma complexidade temporal que varia conforme o caso:

Complexidade dominante: $O(\log k + n)$ no caso médio e melhor caso, $O(k)$ no pior caso

- Onde k = número total de estações (64.000+ no dataset real)
- n = número de vizinhos solicitados pelo usuário

Análise detalhada:

- Melhor caso: Ocorre quando o pruning é altamente eficaz, explorando apenas $O(\log k)$ nós da árvore e processando os n vizinhos encontrados
- Caso médio: O algoritmo explora $O(\log k)$ nós e processa os n resultados, resultando em $O(\log k + n)$
- Pior caso: Ocorre quando pouco pruning é possível (todas as estações em distâncias similares), resultando em $O(k)$ onde k é o número total de estações